



**CENTRO UNIVERSITÁRIO – CATÓLICA DE SANTA
CATARINA
ENGENHARIA DE SOFTWARE**

**RELATÓRIO
DESAFIOS DE CODE INTERVIEW**

Alunos:

Alexandre Sebastian Muller

Mariele Vieira da Silva

Maria Alice Giuliari

1. Escolha da IA para Desenvolvimento

Ferramenta Selecionada

A equipe optou pelo Gemini e ChatGPT como ferramenta principal de apoio ao desenvolvimento desta atividade.

Forma de Acesso

A ferramenta foi acessada via interface Web, disponível no endereço gemini.google.com, [ChatGPT](https://chatgpt.com), utilizando as contas pessoais dos integrantes da equipe.

Justificativa da Escolha

A escolha do Gemini e do ChatGPT se deu principalmente por conta da familiaridade de todos os membros da equipe com a ferramenta, o que facilitou sua utilização ao longo do desenvolvimento. Além disso, esta ferramenta oferece suporte à geração de código, explicações conceituais e auxílio na identificação de erros, características que foram muito importantes para a realização da atividade proposta.

Materiais de Apoio

Não foram utilizados tutoriais ou cursos específicos para o uso da ferramenta, uma vez que todos já possuíam experiência prévia. A ferramenta também não foi integrada diretamente a uma IDE, sendo utilizada exclusivamente via navegador.

Metodologia de Uso (Workflow)

A equipe utilizou a IA como suporte em diferentes etapas do desenvolvimento, incluindo:

- Geração inicial de ideias de solução
- Explicação de conceitos e estruturas de dados
- Revisão de código e identificação de possíveis erros
- Sugestão de casos de teste adicionais

A utilização da ferramenta não substituiu o raciocínio da equipe, sendo as respostas analisadas antes de sua aplicação.

2. Inventário de Problemas

Problemas de Arrays		
Nome do Problema	Nível do Problema	Explicação Breve
Product of Array Except Self	Médio	Criar um novo array onde cada posição contém o produto de todos os outros elementos, exceto o da própria posição.
Container With Most Water	Médio	Determinar a maior área possível de água entre duas linhas verticais representadas em um array.

Two Sum	Fácil	Encontrar dois índices de elementos cuja soma seja igual a um valor alvo.
Best Time to Buy and Sell Stock	Fácil	Maximizar o lucro escolhendo um único dia para comprar uma ação e outro dia, em uma data futura diferente, para vendê-la.
Maximum Subarray	Médio	Encontre o sub-array com a maior soma, e retorne sua soma .
Problemas de Listas Encadeadas		
Nome do Problema	Nível do Problema	Explicação Breve
Add Two Numbers	Médio	Somar dois números representados por listas encadeadas e retornar o resultado como uma nova lista.
Reverse Linked List	Fácil	Inverter a ordem dos nós de uma lista encadeada.
Remove Nth Node From End of List	Médio	Remover o n-ésimo nó a partir do final de uma lista encadeada e retornar a lista atualizada.
Linked List Cycle	Fácil	Dado o primeiro elemento de uma lista encadeada, determine se a lista encadeada contém um ciclo.
Problemas de Pilhas		
Nome do Problema	Nível do Problema	Explicação Breve
Daily Temperatures	Médio	Determinar, para cada dia, quantos dias devem passar até uma temperatura maior ocorrer.
Simplify Path	Médio	Converter um caminho de diretório com “.” e “..” e redundâncias em um caminho canônico.
Valid Parentheses	Fácil	Dada uma string contendo apenas os caracteres '(', ')', '{', '}', '[' e ']', determine se a string de entrada é válida.

3. Implementação e Explicação dos Problemas

Product of Array Except Self

- Plataforma: LeetCode
- Link: [Product of Array Except Self - LeetCode](#)
- Problema de Array

Descrição do Problema:

O problema Product of Array Except Self consiste em, dado um vetor de inteiros nums de tamanho n (com $n \geq 2$), construir um novo vetor answer também de tamanho n, no qual cada posição i contém o produto de todos os elementos de nums, exceto o elemento localizado no próprio índice i.

Estratégia da Solução:

Inicialmente, cria-se um vetor resp preenchido com 1 e percorre-se o array da esquerda para a direita, armazenando em cada posição o produto dos elementos anteriores por meio de um acumulador (esq). Em seguida, realiza-se um segundo percurso da direita para a esquerda, utilizando outro acumulador (der) que representa o produto dos elementos posteriores, multiplicando-o ao valor já presente em answer[i]. Dessa forma, cada posição passa a conter o produto de todos os elementos do vetor, exceto o próprio.

```
nums3 = [-1,1,0,-3,3]
nums = [4,6,8,2,8]
nums2 = [1,2,3,4]

resp = [1] * len(nums)

for i in range(len(nums)):
    if i < len(nums) - 1:
        ini = nums[i] * nums[i + 1]
        resp[i] = ini

for i in range(len(nums) - 1, -1, -1):
    fim = nums[i] * nums[i - 1]
    resp[i] = resp[i] * fim

print(resp)
```

Código da Solução Inicial

Uso de IA (Prompt 1):

- **Pergunta:** Eu gostaria que me ajudasse a resolver esse problema, apenas me dê dicas e explique exatamente o que está sendo exigido.
- **Motivação:** A resposta obtida com a primeira versão do código não apresentou a solução esperada, então busco auxílio para resolver.
- **Resposta da IA:** Ela sugeriu manter a abordagem de multiplicação cruzada, mas que também mudasse um pouco a lógica, utilizando uma constante para substituir o primeiro valor.

1. Boa, esse é um clássico de entrevistas 🧐 — vamos focar **em entender exatamente o que o problema pede** e depois te guiar com dicas (sem entregar direto a solução).

2. 💡 DICA PRINCIPAL (essa é a chave 🔑)

Para cada posição i , você precisa de:

produto dos elementos à ESQUERDA de i

produto dos elementos à DIREITA de i

🔍 Visualizando melhor

Para:

`nums = [1,2,3,4]`

Produtos da esquerda:

`[1, 1, 2, 6]`

(explicação)

posição 0 → nada à esquerda → 1

posição 1 → 1

posição 2 → $1 \cdot 2 = 2$

posição 3 → $1 \cdot 2 \cdot 3 = 6$

- **Avaliação:** A proposta sugerida pela IA se mostrou eficiente, e esclareceu o erro que estava ocorrendo com a lógica principal.

```
n = len(nums)
resp = [1] * n

esq = 1
for i in range(n):
    resp[i] = esq
    esq *= nums[i]

der = 1
for i in range(n - 1, -1, -1):
    resp[i] *= der
    der *= nums[i]

print(resp)
```

Código segunda versão

```
32 print("Desafio proposto pelo LeetCode: https://leetcode.com/problems/product-of-array-except-s
33 print("output esperado: [24, 12, 8, 6]")
34 print("output obtido: ", resp)
35
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [] [X] ... | [C] X

```
PS C:\projetos\pasta para atividades (temporarias)> & C:/Users/Lexan/AppData/Local/Microsoft/windowsApps/python3.11.exe "c:/projetos/pasta para atividades (temporarias)/product-of-array-except-self.py"
Desafio proposto pelo LeetCode: https://leetcode.com/problems/product-of-array-except-self/
output esperado: [24, 12, 8, 6]
output obtido: [24, 12, 8, 6]
PS C:\projetos\pasta para atividades (temporarias)>
```

Identificando a Complexidade do Código:

Complexidade de Tempo

A complexidade de tempo desse código é definido pelo loop que percorre o array duas vezes, sendo assim **$O(n)$** , onde cada loop percorre “n” elementos, sendo assim caso o número de elementos no array dobre, o tempo também vai dobrar, sendo um crescimento linear.

Complexidade de Memória

Pela exigência de criar um array exclusivo para respostas faz com que a nossa complexidade de memória também seja **$O(n)$** um crescimento linear, aumentando o seu valor junto de do array **nums**, mas caso não se considere o array adicional, e apenas as variáveis auxiliar, a complexidade de memória seria **$O(1)$ – Constante**.

Justificativa da Estrutura de Dados:

- O array `extra` é utilizado para armazenar a resposta da questão sem alterar os valores originais de `nums`.

- Ambas as variáveis “esq” e “der” são utilizadas para manter o valor inicial do primeiro campo, assim obtendo o produto de todos os elementos dentro de **nums** sem contar a si mesmo.

Testes

Além dos testes oferecidos pelo desafio, foram feitos alguns outros tentando encontrar algum comportamento peculiar.

- Números negativos:

```
conjunto com numeros negativos  
input: [-1, 1, 0, -3, 3]  
output: [0, 0, 9, 0, 0]
```

- Uma string em meio ao conjunto de números inteiros:

```
conjunto com string  
input: ['1', 2, 3, 4]  
output: [24, '111111111111', '11111111', '111111']
```

- Números quebrados:

```
conjunto de numeros quebrados  
input: [2.2, 2.5, 2.5, 2.5]  
output: [15.625, 13.750000000000002, 13.75, 13.75]
```

Container With Most Water

- Plataforma: LeetCode
- Link: [Container With Most Water - LeetCode](#)
- Problema de Array

Descrição do Problema:

O problema Container With Most Water consiste em dado um array de inteiros onde cada elemento representa a altura de uma linha vertical, encontrar duas linhas que, junto com o eixo horizontal, formem um recipiente capaz de armazenar a maior quantidade de água possível; a quantidade de água é determinada pela menor altura entre as duas linhas multiplicada pela distância entre elas, e o desafio é escolher o par de linhas que maximize essa área sem inclinar o recipiente.

Estratégia da Solução:

Definir uma variável para obter os valores máximos e mínimos da distância e de comprimento dos valores dentro do array height, assim resolvendo o problema.

```
1 height = [1,8,6,2,5,4,8,3,7]
2 height2 = [1,1]
3 area = 0 #altura * distancia
4
5 n = len(height)
6 for i in range(n):
7     for j in range(n):
8         if i == j:
9             continue
10        dis = min(height[i] , height[j]) * (max(i, j) - min(j, i))
11        if dis > area:
12            area = dis
```

Primeira Versão do Código

Uso de IA (Prompt 1):

- Pergunta:

de quais maneiras eu posso otimizar esse código

Seu código funciona, mas está com complexidade $O(n^2)$ por causa dos dois loops. Dá pra otimizar bastante — inclusive chegar em $O(n)$.

🚀 1. Primeira otimização (ainda $O(n^2)$, mas melhor)

Você não precisa comparar todos os pares duas vezes nem comparar `i == j`.

👉 Basta fazer `j` começar de `i + 1`:

```
</> Python
for i in range(n):
    for j in range(i + 1, n):
        area = max(area, min(height[i], height[j]) * (j - i))
```



▶ Executar

- ✓ Evita comparações repetidas
- ✓ Remove o `if i == j`
- ✓ Mais limpo



- **Motivação:** Após a resolução do problema o nível de complexidade é $O(n^2)$ acredito que é possível diminuir esse valor.
- **Resposta da IA:**

⚡ 2. Otimização ideal (Two Pointers → $O(n)$)

Essa é a solução mais importante (e a esperada em entrevistas tipo LeetCode).

💡 Ideia:

- Começa com dois ponteiros:
 - um no início (`left`)
 - outro no final (`right`)
- Calcula a área
- Move o ponteiro da menor altura

```
area = 0
n = len(height)
n = len(height)
for i in range(n):
    for j in range(i + 1, n):
        dis = max(area, min(height[i], height[j]) * (j - i))
        if dis > area:
            area = dis
```

Segunda Versão Otimizando for

- **Avaliação da Resposta:** Boa, o processo de otimização conseguiu diminuir o tempo de execução, mas a complexidade continua sendo $O(n^2)$.

```
left = 0
right = len(height) - 1
area = 0

while left < right:
    h = min(height[left], height[right])
    area = max(area, h * (right - left))

    if height[left] < height[right]:
        left += 1
    else:
        right -= 1
```

Terceira Versão Otimizado

- **Avaliação da resposta:** Excelente, a otimização foi extremamente eficiente, não somente reduzindo o tempo de compilação mas também tornando o nível de complexidade $O(n^2)$.

Identificando a Complexidade do Código:

Complexidade de Tempo

A solução com dois ponteiros percorre o array apenas uma vez, movendo `left` ou `right` a cada iteração, o que garante que cada elemento seja visitado no máximo uma vez; por isso, a complexidade de tempo é $O(n)$, ou seja, cresce linearmente conforme o tamanho do array aumenta.

Complexidade de Memória

A solução utiliza apenas algumas variáveis auxiliares (`left`, `right`, `area`, `h`) e não cria nenhuma estrutura adicional proporcional ao tamanho da entrada, portanto a complexidade de memória é $O(1)$, ou seja, constante.

Testes

- Teste com números negativos:

```
teste numeros negativos: [-4, 3, 2, 1, 4]
9
```

- Teste com números quebrados:

```
teste numeros quebrados: [7.7, 3.3, 2.2, 1.1, 4.4]
17.6
```

- Teste com string entre os números:

```
h = min(height[left], height[right])
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: '<' not supported between instances of 'int' and 'str'
```

- O teste foi falho pois não é possível comparar uma string com um int.

Justificativa da Estrutura de Dados:

- As variáveis `left` e `right` são utilizadas para a navegação entre containers.
- `while` é executado até a proximidade máxima dos containers.
- `area` é utilizada em seu próprio cálculo para se obter a maior possível.

Two Sum

- Plataforma: LeetCode
- Link: [Two Sum - LeetCode](#)
- Problema de Array

Descrição do Problema:

Dado um array de nums (números) inteiros e um valor alvo (o target) inteiro, o objetivo é retornar os índices de dois elementos cuja soma seja igual ao valor alvo.

Assume-se que existe exatamente apenas uma solução para cada input, e que o mesmo elemento não pode ser utilizado duas vezes, e também é possível retornar as respostas em qualquer ordem.

Estratégia da Solução:

Inicialmente, foi implementada uma solução utilizando dois loops aninhados, onde cada elemento do array é comparado com os demais em busca de uma combinação válida.

Posteriormente, a solução foi otimizada com o uso de um Hash Map (dicionário), armazenando os valores já percorridos e seus respectivos índices. O Hash Map é utilizado para armazenar os valores já percorridos e seus respectivos índices, permitindo verificar rapidamente se o complemento necessário já foi encontrado anteriormente, reduzindo a complexidade da solução.

```
# Dado um array de nums (números) inteiros e um target (alvo) inteiro, retornar índices dos dois numeros cuja soma seja igual ao target, sem repetir elementos

nums = [5, 2, 39, 17, 30, 4, 9, 20]
target = 32
|
print('Array:', nums)
print('Target:', target)
def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)): # Corrigido: o loop interno deve começar de i + 1
            if nums[i] + nums[j] == target:
                return [i, j]

resultado = twoSum(nums, target)

print('Índices dos dois números:', resultado)
```

```
... Array: [5, 2, 39, 17, 30, 4, 9, 20]
Target: 32
Índices dos dois números: [1, 4]
```

Código da Solução Inicial

Uso de IA (Prompt 1):

- Pergunta:

Considerando o seguinte código:

```
# Dado um array de nums (números) inteiros e um target (alvo) inteiro,
retornar índices dos dois números cuja soma seja igual ao target, sem
repetir elementos
```

```
nums = [5, 2, 39, 17, 30, 4, 9, 20]
target = 32
```

```
print('Array:', nums)
print('Target:', target)
```

```
def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)): # Corrigido: o loop interno deve
            começar de i + 1
            if nums[i] + nums[j] == target:
                return [i, j]
```

```
resultado = twoSum(nums, target)
```

```
print('Índices dos dois índices:', resultado)
```

Como posso alterá-lo para que o usuário digite os números do array e o target via input()?

- **Motivação:** Permitir testes dinâmicos com valores diferentes.
- **Resposta da IA:** Sugeriu o uso de `input()`, `split()` e `map(int, ...)` para converter os dados.

1. Ler o array com `input()` (normalmente separado por espaço ou vírgula).
2. Converter os valores para `int`.
3. Ler o target com `input()` e converter para `int`.

A forma mais simples é pedir os números separados por espaço.

Versão modificada do seu código

Python



Executar

```
# Dado um array de nums (números) inteiros e um target (alvo) inteiro,
# retornar índices dos dois números cuja soma seja igual ao target

nums = list(map(int, input("Digite os números do array separados por espaço: ").split()))
target = int(input("Digite o target: "))

print('Array:', nums)
print('Target:', target)

def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]

resultado = twoSum(nums, target)

print('Índices dos dois números:', resultado)
```

Exemplo de execução

Entrada do usuário:

```
Digite os números do array separados por espaço: 5 2 39 17 30 4 9 20
Digite o target: 32
```

Saída:

```
Array: [5, 2, 39, 17, 30, 4, 9, 20]
Target: 32
Índices dos dois números: [1, 4]
```

💡 Explicação rápida da parte nova:

```
</> Python
nums = list(map(int, input().split()))
```

- `input()` → lê a linha digitada
- `.split()` → separa pelos espaços
- `map(int, ...)` → transforma cada valor em inteiro
- `list()` → transforma em lista

- **Avaliação:** A resposta foi correta e utilizada sem necessidade de ajustes.

```
# Dado um array de nums (números) inteiros e um target (alvo) inteiro,
# retornar índices dos dois números cuja soma seja igual ao target

nums = list(map(int, input("Digite os números do array separados por espaço: ").split()))
target = int(input("Digite o target: "))

print('Array:', nums)
print('Target:', target)

def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]

resultado = twoSum(nums, target)

print('Índices dos dois números:', resultado)
```

```
... Digite os números do array separados por espaço: 4 9 3 2
Digite o target: 11
Array: [4, 9, 3, 2]
Target: 11
Índices dos dois números: [1, 3]
```

Código com entrada via input

Identificando a Complexidade do Código:

Complexidade de Tempo

O código está utilizando dois loops `for`, um dentro do outro. Isto significa que, para cada número na lista, o código está percorrendo a lista inteira novamente procurando o par. Isto pode se tornar um problema, já que, caso tivermos um array de 10 números, seriam feitas até 100 comparações, por exemplo.

Esta é a **Complexidade Quadrática ($O(n^2)$)**, funcional para listas pequenas, porém problemático quando estamos trabalhando com grandes volumes de dados.

Complexidade de Memória

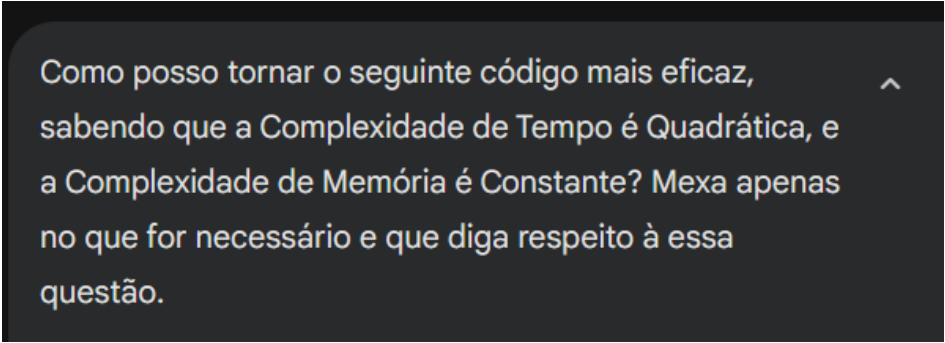
Como neste código estamos apenas usando duas variáveis (*i* e *j*) para controlar os índices, e não criando novas listas ou estruturas que guardam todos os dados novamente, o espaço usado na memória é fixo, não importando se o array tiver 10 ou 1000 itens.

Portanto, esta é a **Complexidade Constante ($O(1)$)**.

Assim, podemos dizer que o código é lento (Tempo), porém econômico (Memória).

Uso de IA (Prompt 2):

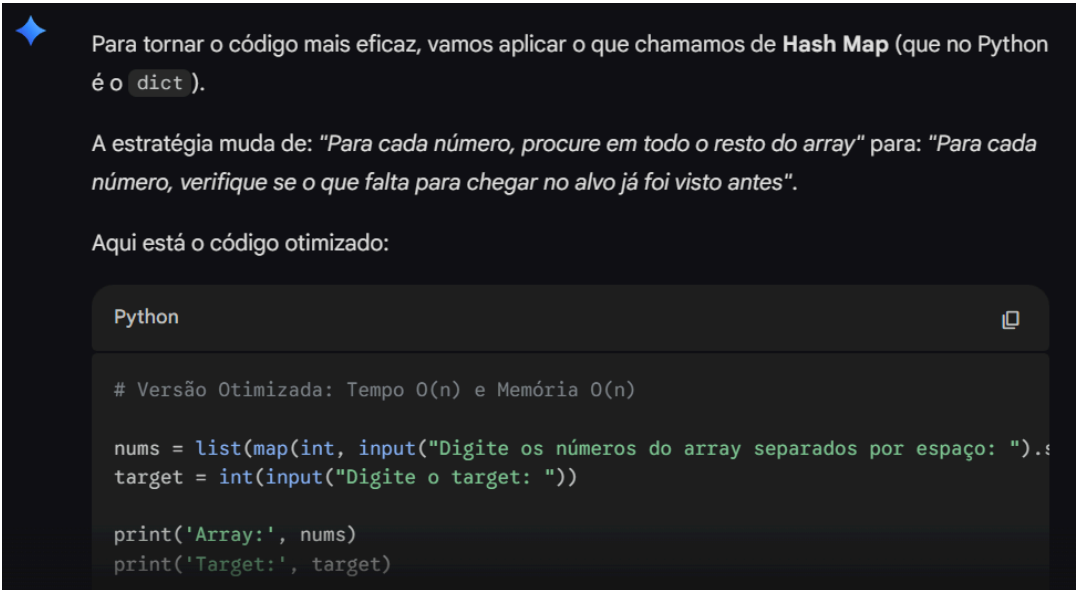
- **Pergunta:**



Como posso tornar o seguinte código mais eficaz, sabendo que a Complexidade de Tempo é Quadrática, e a Complexidade de Memória é Constante? Mexa apenas no que for necessário e que diga respeito à essa questão.

2º Prompt

- **Motivação:** Reduzir a complexidade de tempo da solução inicial, que apresentava complexidade quadrática.
- **Resposta da IA:** Sugeriu o uso de um Hash Map para armazenar valores já percorridos.



✦ Para tornar o código mais eficaz, vamos aplicar o que chamamos de **Hash Map** (que no Python é o `dict`).

A estratégia muda de: "Para cada número, procure em todo o resto do array" para: "Para cada número, verifique se o que falta para chegar no alvo já foi visto antes".

Aqui está o código otimizado:

```
Python

# Versão Otimizada: Tempo O(n) e Memória O(n)

nums = list(map(int, input("Digite os números do array separados por espaço: ").split()))
target = int(input("Digite o target: "))

print('Array:', nums)
print('Target:', target)
```

O que mudou e por que ficou melhor?

1. **Apenas um Loop ($O(n)$):** Agora percorremos a lista apenas uma vez. Em vez de um segundo `for` (que era lento), fazemos uma busca no dicionário `if complemento in mapa_indices`.
2. **Busca Instantânea:** Em Python, verificar se uma chave existe em um dicionário leva tempo constante ($O(1)$). É por isso que o tempo total cai de quadrático para **linear**.
3. **O "Trade-off" de Memória:** Como agora usamos o `mapa_indices` para memorizar os números que já passaram, a complexidade de memória subiu para $O(n)$. Sacrificamos um pouco de RAM para ganhar muita velocidade de processamento.

- **Avaliação:** A solução foi adotada com sucesso, tendo a complexidade de tempo reduzida para $O(n)$.

```
# Versão Otimizada: Tempo  $O(n)$  e Memória  $O(n)$ 

nums = list(map(int, input("Digite os números do array separados por espaço: ").split()))
target = int(input("Digite o target: "))

print('Array:', nums)
print('Target:', target)

def twoSum(nums, target):
    # Dicionário para guardar {valor: indice}
    mapa_indices = {}

    for i, num in enumerate(nums):
        complemento = target - num

        # Se o complemento já estiver no dicionário, achamos a dupla!
        if complemento in mapa_indices:
            return [mapa_indices[complemento], i]

        # Caso contrário, guarda o número atual e seu índice no dicionário
        mapa_indices[num] = i

    resultado = twoSum(nums, target)

    print('Índices dos dois números:', resultado)
```

*** Digite os números do array separados por espaço: 4 3 10 48
Digite o target: 7
Array: [4, 3, 10, 48]
Target: 7
Índices dos dois números: [0, 1]

Código Otimizado com Hash Map

Justificativa da Estrutura de Dados:

- O array é a estrutura base do problema.
- O Hash Map permite buscas em tempo constante, tornando a solução mais rápida e eficiente.

Essa abordagem reduz significativamente o tempo de execução em comparação com a solução inicial.

Testes

Casos Especiais Considerados:

- Elementos repetidos (ex.: 5 + 5).
- Números negativos.
- Casos onde não há solução.
- Arrays com tamanho mínimo.

Teste Manual:

Foram criados testes para validar diferentes cenários:

- Números repetidos.
- Presença de números negativos.
- Caso sem solução.

Foi possível observar que a ordem dos índices retornados pode variar, sem impactar a validade da resposta.

```
# Casos de Teste Adicionais
testes = [
    {"nums": [1, 5, 5, 11], "target": 10, "esperado": [1, 2], "nome": "Números repetidos"},
    {"nums": [10, -2, 5, 8], "target": 8, "esperado": [1, 0], "nome": "Números negativos"},
    {"nums": [1, 2, 3], "target": 7, "esperado": None, "nome": "Sem solução"}
]

for t in testes:
    res = twoSum(t["nums"], t["target"])
    print(f"Teste: {t['nome']} | Resultado Obtido: {res} | Resultado Esperado: {t['esperado']}")

... Teste: Números repetidos | Resultado Obtido: [1, 2] | Resultado Esperado: [1, 2]
Teste: Números negativos | Resultado Obtido: [0, 1] | Resultado Esperado: [1, 0]
Teste: Sem solução | Resultado Obtido: None | Resultado Esperado: None
```

Apesar de no teste de números negativos os índices estarem invertidos (enquanto o resultado esperado seria [1,0], o resultado obtido foi [0,1]), não consideramos um erro, já que a ordem dos índices não alteram o resultado da soma. 🐞

Testes Manuais

Teste Automatizado:

- Feito utilizando a biblioteca unittest do python.

```
import unittest

class TestTwoSum(unittest.TestCase):
    def test_numeros_repetidos(self):
        # Caso onde os números que somam o target são iguais (ex: 5+5=10)
        self.assertEqual(twoSum([1, 5, 5, 11], 10), [1, 2])

    def test_numeros_negativos(self):
        # Caso com números negativos
        self.assertEqual(twoSum([10, -2, 5, 8], 8), [0, 1])

    def test_lista_curta(self):
        # O menor array possível (apenas 2 elementos)
        self.assertEqual(twoSum([3, 3], 6), [0, 1])

# Executa os testes no ambiente do Colab
unittest.main(argv=['first-arg-is-ignored'], exit=False)

...
-----
Ran 3 tests in 0.004s

OK
<unittest.main.TestProgram at 0x7c4d1b4a0350>
```

Testes Automatizados

Best Time to Buy and Sell Stock

- Plataforma: LeetCode
- Link: [Best Time to Buy and Sell Stock - LeetCode](#)
- Problema de Array

Descrição do Problema:

É fornecido um array `prices` onde `prices[i]`, representa o preço de uma determinada ação no dia *ith*. O objetivo é maximizar o lucro escolhendo um único dia para comprar uma ação e outro dia, em uma data futura diferente, para vendê-la. Retorne o lucro máximo que você puder obter com esta transação . Se você não puder obter nenhum lucro, devolva o valor 0.

Estratégia da Solução:

O problema envolve analisar um *array* de preços ao longo do tempo para determinar o maior lucro possível com uma única compra e venda. O uso de array é adequado porque os dados representam uma sequência temporal, permitindo percorrer os valores uma única vez e manter o menor preço encontrado até o momento, calculando o lucro máximo de forma eficiente ($O(n)$).

```
# Sem IA
from typing import List

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        min_preco = prices[0]
        max_lucro = 0

        for preco in prices:
            if preco < min_preco:
                min_preco = preco
            else:
                max_lucro = max(max_lucro, preco - min_preco)

            min_preco = min(min_preco, preco)

        return max_lucro
```

Código da Solução Inicial

Uso de IA (Prompt 1):

- **Pergunta:** “Para resolver esse problema [...], com esses exemplos de entrada e saída [...] o código abaixo corresponde com exatidão à solução? [...]”
- **Motivação:** Encontrar a melhor solução possível para resolução do problema.
- **Resposta da IA:**

 **O problema no seu código**

Você tem isso dentro do loop:

```
</> Python   Executar
```

```
if preco < min_preco:
    min_preco = preco
else:
    max_lucro = max(max_lucro, preco - min_preco)

min_preco = min(min_preco, preco)
```

👉 Essa última linha:

```
</> Python   Executar
```

```
min_preco = min(min_preco, preco)
```

é desnecessária, porque você já tratou isso no `if`.

- **Versão Corrigida do Código:**

 **Versão corrigida (mais limpa)**

```
</> Python   Executar
```

```
from typing import List

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        min_preco = prices[0]
        max_lucro = 0

        for preco in prices:
            if preco < min_preco:
                min_preco = preco
            else:
                max_lucro = max(max_lucro, preco - min_preco)

        return max_lucro
```

- **Avaliação:** Além da correção de código a IA me trouxe outra versão mais profissional e utilizada em entrevistas, para comparação e também me deu uma explicação do raciocínio do problema o que foi muito interessante para a compreensão mental do problema, fugindo da codificação apenas.

💡 Intuição (o segredo da questão)

Você está basicamente fazendo:

- Guardar o menor preço até agora
- A cada dia, calcular:

```
lucro = preço_atual - menor_preço_anterior
```



- Atualizar o maior lucro

🔧 Teste mental (seu exemplo)

[7,1,5,3,6,4]

Dia	Preço	min_preco	lucro	max_lucro	📄
1	7	7	0	0	
2	1	1	0	0	
3	5	1	4	4	
4	3	1	2	4	
5	6	1	5	5	
6	4	1	3	5	

Código com Entrada via input

Identificando a Complexidade do Código:

Complexidade de Tempo

A complexidade de tempo é **$O(n)$** , é percorrido a lista prices **uma única vez**. Cada operação dentro do for é **$O(1)$** (comparações e cálculos simples), ou seja, se o array tem n elementos → você faz n iterações.

Tempo = $O(n)$

Complexidade de Memória

Como neste código estamos apenas usando **$O(1)$** (constante), pois é utilizado apenas variáveis simples min_preco, max_lucro e preco.

Portanto, não importa o tamanho da entrada, você **não cria estruturas adicionais** (como listas, dicionários, etc.).

Memória = $O(1)$

Uso de IA (Prompt 2):

- **Pergunta:** Versão mais profissional utilizada em entrevistas.

Essa aqui é a versão mais "profissional":

</> Python



▶ Executar

```
from typing import List

class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        min_preco = float('inf')
        max_lucro = 0

        for preco in prices:
            min_preco = min(min_preco, preco)
            max_lucro = max(max_lucro, preco - min_preco)

        return max_lucro
```

2º Resposta

- **Motivação:** Reduzir a complexidade de tempo da solução inicial, deixando-a mais elegante.
- **Avaliação:** A solução foi adotada com sucesso, tendo a complexidade de tempo reduzida para $O(n)$.

Justificativa da Estrutura de Dados:

- Acesso sequencial eficiente: o problema exige percorrer os preços em ordem cronológica, e a lista permite interação direta e eficiente ($O(n)$).
- Acesso rápido aos elementos: cada elemento pode ser acessado em tempo constante ($O(1)$), o que é ideal para operações simples como comparação e cálculo de lucro.
- Simplicidade e baixo custo: não há necessidade de estruturas mais complexas (como pilhas, filas ou árvores), pois o problema pode ser resolvido apenas com uma varredura linear.
- Uso mínimo de memória adicional: o algoritmo não cria novas estruturas, apenas utiliza variáveis auxiliares (`min_preco` e `max_lucro`), mantendo o uso de memória constante.

Testes

Casos Especiais Considerados:

- Venda antes da compra.
- Lucro máximo igual a 0(zero).

Teste Manual:

DescriçãoEditorialSoluçõesSubmissões

É fornecido um array `prices` onde `prices[i]` `price` representa o preço de uma determinada ação no dia `ith`.

O objetivo é maximizar o lucro escolhendo um **único dia** para comprar uma ação e outro **dia, em uma data futura diferente**, para vendê-la.

Retorne o *lucro máximo* que você puder obter com esta transação . Se você não puder obter nenhum lucro, devolva o valor `0`.

Exemplo 1:

Entrada: `preços = [7,1,5,3,6,4]`
Saída: `5`
Explicação: Compre no dia 2 (preço = 1) e venda no dia 5 (preço = 6), lucro = 6-1 = 5.
Observe que comprar no dia 2 e vender no dia 1 não é permitido, pois você deve comprar antes de vender.

Exemplo 2:

Entrada: `preços = [7,6,4,3,1]`
Saída: `0`
Explicação: Neste caso, nenhuma transação é feita e o lucro máximo = 0.

Restrições:

Código

Python3Auto

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        min_preco = float('inf')
        max_lucro = 0

        for preco in prices:
            min_preco = min(min_preco, preco)
            max_lucro = max(max_lucro, preco - min_preco)

        return max_lucro
```

Salvo

Caso De TesteResultado Do Teste

Aceito Tempo de execução: 0 ms

Caso 1

Caso 2

Entrada

preços =
[7,1,5,3,6,4]

Saída

5

Testes Manuais

Maximum Subarray

- Plataforma: LeetCode
- Link: [Maximum Subarray - LeetCode](#)
- Problema de Array

Descrição do Problema:

Dado um array de inteiros nums, encontre o sub-array com a maior soma, e retorne sua soma.

Estratégia da Solução:

A ideia central é percorrer o array **uma única vez**, decidindo a cada posição se deve continuar o sub-array ou iniciar um novo.

```
# Sem IA
from typing import List

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        n = len(nums)
        maior_soma = nums[0]

        for i in range(n):
            soma_atual = 0
            for j in range(i, n):
                soma_atual += nums[j]

            if soma_atual > maior_soma:
                maior_soma = soma_atual

        return maior_soma
```

Código da Solução Inicial

Uso de IA (Prompt 1):

- **Pergunta:** “Para resolver esse problema [...], com esses exemplos de entrada e saída [...]. Essa aqui é a minha versão com solução do problema [...], ainda não realizei testes mas queria saber se é a ideal.”
- **Motivação:** Encontrar a solução mais eficaz.
- **Resposta da IA:**

seu código atual está correto, mas tem complexidade $O(n^2)$, porque você testa todos os subarrays possíveis. Dá pra melhorar MUITO usando o famoso **Algoritmo de Kadane**, que resolve em $O(n)$ (uma única passada no array).

💡 Ideia principal (bem intuitiva)

Você percorre o array acumulando a soma:

- Se a soma atual ficar ruim (negativa), você **descarta e recomeça**
- Sempre guarda o maior valor já encontrado

• Versão corrigida:

 **Versão eficiente ($O(n)$)**

```
Python
from typing import List

class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        soma_atual = nums[0]
        maior_soma = nums[0]

        for i in range(1, len(nums)):
            # decide: continua somando ou começa do zero?
            soma_atual = max(nums[i], soma_atual + nums[i])

            # atualiza o maior valor encontrado
            maior_soma = max(maior_soma, soma_atual)

        return maior_soma
```

Executar

- **Avaliação:** A IA me trouxe uma forma de resolução que mesmo sendo famosa eu ainda não conhecia (Algoritmo de Kadane), o que me trouxe uma nova visão do problema e que com certeza irei utilizar em outros códigos.

“A estratégia consiste em acumular a soma do sub-array enquanto for vantajoso e reiniciar quando ela se torna prejudicial, mantendo sempre o maior valor encontrado.”

Para cada número você decide:

👉 "Vale a pena continuar o subarray anterior ou começar um novo daqui?"

Isso é essa linha:

Python

```
soma_atual = max(nums[i], soma_atual + nums[i])
```



Executar

⚡ Comparação

Abordagem	Complexidade	📄
Sua (for duplo)	$O(n^2)$	
Kadane	$O(n)$	

Explicação da Solução

Identificando a Complexidade do Código:

Complexidade de Tempo

O algoritmo percorre o array apenas uma vez, em cada iteração, realiza operações constantes de soma e comparação (max). Ou seja, o número de operações cresce linearmente com o tamanho do array. Assim, a complexidade de tempo é **$O(n)$** (linear).

Complexidade de Memória

A complexidade de memória é **$O(1)$** (constante). Utiliza apenas variáveis simples: `soma_atual` e `maior_soma`. Não cria estruturas auxiliares (listas, matrizes, etc.). O uso de memória **não depende do tamanho da entrada**.

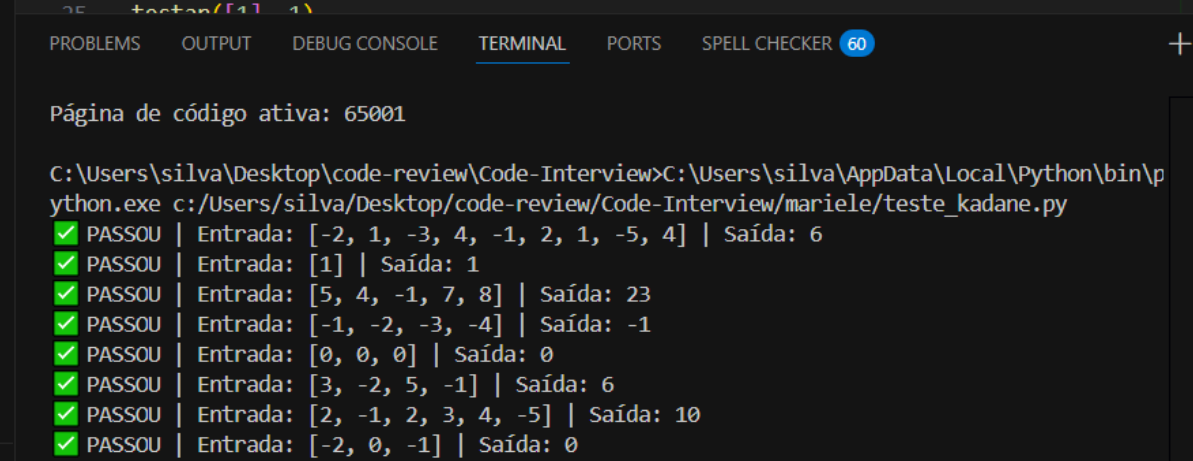
Justificativa da Estrutura de Dados:

- **Acesso direto e sequencial ($O(1)$):**
Cada elemento pode ser acessado rapidamente por índice, permitindo a atualização contínua da soma atual.
- **Preservação da ordem dos elementos:**
Como o problema envolve **subarrays contínuos**, é fundamental manter a ordem original dos dados — algo garantido pelo array.
- **Eficiência na iteração:**
O algoritmo percorre o array apenas uma vez, e essa estrutura permite uma iteração linear sem custos adicionais.
- **Baixo uso de memória:**
O Kadane não exige estruturas auxiliares, apenas variáveis simples, aproveitando o array já fornecido como entrada.

Testes

Casos Especiais Considerados:

- Casos básicos.
- Casos com números negativos.
- Casos com zeros.
- Casos mistos.
- Casos de borda.
- Teste grande (performance).



A screenshot of a terminal window with a dark background. The terminal title bar shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (active), PORTS, and SPELL CHECKER (60). The text in the terminal reads: 'Página de código ativa: 65001' followed by a command to run a Python script. Below the command, there are eight lines of test results, each starting with a green checkmark and the word 'PASSOU'. Each line shows an input array, the output value, and the word 'Saída'.

```
Página de código ativa: 65001
C:\Users\silva\Desktop\code-review\Code-Interview>C:\Users\silva\AppData\Local\Python\bin\python.exe c:/Users/silva/Desktop/code-review/Code-Interview/mariele/teste_kadane.py
✓ PASSOU | Entrada: [-2, 1, -3, 4, -1, 2, 1, -5, 4] | Saída: 6
✓ PASSOU | Entrada: [1] | Saída: 1
✓ PASSOU | Entrada: [5, 4, -1, 7, 8] | Saída: 23
✓ PASSOU | Entrada: [-1, -2, -3, -4] | Saída: -1
✓ PASSOU | Entrada: [0, 0, 0] | Saída: 0
✓ PASSOU | Entrada: [3, -2, 5, -1] | Saída: 6
✓ PASSOU | Entrada: [2, -1, 2, 3, 4, -5] | Saída: 10
✓ PASSOU | Entrada: [-2, 0, -1] | Saída: 0
```

Testes Manuais

Add-two-numbers

- Plataforma: LeetCode
- Link: [Add Two Numbers - LeetCode](#)
- Problema de Lista

Descrição do Problema:

Deve-se somar dois números representados por listas ligadas, onde cada nó contém um único dígito e os dígitos estão armazenados em ordem reversa; o objetivo é retornar a soma também como uma lista ligada, mas armazenando apenas um valor em cada nó.

Estratégia da Solução:

Somar cada nó dentro de ambas as listas e caso o valor seja igual ou maior que 10, utilizar uma variável para incrementar na soma seguinte, e subtraindo 10 do valor total para armazenar apenas uma variável por nó.

```
l1 = [2, 4, 3]
l2 = [5, 6, 4]
l3 = []
add = 0

while l1 is not None or l2 is not None:
    soma = l1.val + l2.val + add
    if l1 is not None:
        l1 = l1.next
    if l2 is not None:
        l2 = l2.next
    if soma >= 10:
        add = 1
        soma = soma % 10
        l3.append(soma)
    else:
        l3.append(soma)
print(l3)
```

Primeira Versão Falha

Uso de IA (Prompt 1):

- Pergunta:

o código fornecido apresenta falhas na manipulação da lista, apresente as seguintes falhas junto de uma descrição do problema

Seu código tem a ideia certa, mas existem alguns problemas importantes na manipulação das listas ligadas. Aqui vão as falhas principais com explicação:

- **Motivação:** Compreender as falhas cometidas e melhorar o entendimento de manipulação de listas.
- **Resposta da IA:** Ela identificou que a lógica do código está correta, mas não trata corretamente a manipulação de listas com funcionalidades do Python.

✗ 1. Acesso a `l1.val` e `l2.val` sem verificar se são `None`

Você faz isso logo no começo do loop:

<> Python

```
soma = l1.val + l2.val + add
```



▶ Executar

● Problema:

Se `l1` ou `l2` forem `None`, isso vai gerar erro (`AttributeError`), porque você está tentando acessar `.val` de algo que não existe.

👉 Correto seria tratar assim:

<> Python

```
val1 = l1.val if l1 else 0  
val2 = l2.val if l2 else 0  
soma = val1 + val2 + add
```



▶ Executar

✗ 2. Não trata listas de tamanhos diferentes corretamente

O `while` permite que uma lista acabe antes da outra ↓

- **Avaliação:** Intermediário, mesmo com a explicação ainda não foi possível obter o resultado esperado por conta da manipulação incorreta de lista.

Uso de IA (Prompt 1):

- **Pergunta:**

gostaria que fosse como o problema add two numbers exige, com listas ligadas, como seria ?

Perfeito — agora vamos fazer do jeito que o problema pede, usando listas ligadas de verdade (igual ao padrão do [LeetCode](#)).

- **Motivação:** Entender uma nova metodologia de manipulação de lista com Python.
- **Resposta da IA:** Foi mostrado exemplos claros de como manipular e tratar uma lista, assim como defini-la, dando demonstrações claras.

🟡 Estrutura da lista ligada

Cada nó tem:

- `val` → valor
- `next` → ponteiro para o próximo nó

`</>` Python



▶ Executar

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

- **Avaliação:** Excelente, agora com uma compreensão melhorada do tratamento de lista foi possível finalizar o desafio.

```

class ListNode(object):
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution(object):
    def addTwoNumbers(self, l1, l2):
        dummy = ListNode(0)
        current = dummy
        add = 0

        while l1 or l2 or add:
            val1 = l1.val if l1 else 0
            val2 = l2.val if l2 else 0

            soma = val1 + val2 + add
            add = soma // 10

            current.next = ListNode(soma % 10)
            current = current.next

            if l1:
                l1 = l1.next
            if l2:
                l2 = l2.next

        return dummy.next

def to_list(node):
    result = []
    while node:
        result.append(node.val)
        node = node.next
    return result

# Criar l1: 2 -> 4 -> 3
#l1 = ListNode(2, ListNode(4, ListNode(3)))
# Criar l2: 5 -> 6 -> 4

```

Nova Versão Funcional

Justificativa da Estrutura de Dados:

- `dummy` é utilizado como nossa lista de resposta, podendo ser maior do que as listas comparadas.
- `current` é nosso comparativo de listas, que sempre avança para o próximo valor mesmo que uma das listas já tenham acabado.
- `add` variável adicional responsável pela preservação do valor extrapolado da equação anterior, se somando na próxima operação ou ocupando um espaço próximo caso seja a última variável.

Identificando a Complexidade do Código:

Complexidade de Tempo:

Desconsiderando todos os loops adicionais que só foram criados para possibilitar a realização dos testes no VSCode, a complexidade se mantém $O(n)$ pois para percorrer a lista é utilizado apenas um loop de `while`.

Complexidade de Memória:

Novamente lidando com um problema que exige um retorno em lista(array) o que faz nossa complexidade de memória disparar sendo ele $O(n)$ quanto maior as listas somadas maior será a resposta consequentemente utilizando mais memória, pode-se concluir que nosso programa é rápido mas que consome muita memória.

Testes:

```
Teste com numeros negativos:  
l1: [-2, 4, 3]  
l2: [5, -6, 4]  
Resultado: [3, 8, 6]
```

- Testes realizados com string dão erro imediato pois não é possível executar a operação.
- Demais testes com listas variadas foram um sucesso.

Reverse Linked List

- Plataforma: LeetCode
- Link: [Reverse Linked List - LeetCode](#)
- Problema de Lista Encadeada

Descrição do Problema:

Dado o nó inicial (head) de uma lista encadeada, o objetivo é inverter a ordem dos nós da lista e retornar a lista invertida.

Estratégia da Solução:

A solução foi implementada de forma iterativa, utilizando três ponteiros:

- atual: percorre a lista.
- anterior: armazena o nó anterior.
- proximo: guarda temporariamente o próximo nó.

A cada iteração, o ponteiro do nó atual é invertido, passando a apontar para o nó anterior, e este processo se repete até o final da lista.

A solução inicial apresentou erro devido à posição incorreta do return, que interrompia o loop antecipadamente.

```
# Dado o `head` de uma lista encadeada, inverter a lista e retornar *a lista invertida*.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# criando a lista encadeada
head = ListNode(1)
head.next = ListNode(2)
head.next.next = ListNode(3)
head.next.next.next = ListNode(4)
head.next.next.next.next = ListNode(5)

# função para inverter
def reverseList(head):
    anterior = None
    atual = head

    while atual is not None:
        proximo = atual.next
        atual.next = anterior
        anterior = atual
        atual = proximo
    return anterior

def printLinkedList(head):
    while head is not None:
        print(head.val, end=" -> ")
        head = head.next
    print("None")

print("Lista encadeada original:")
printLinkedList(head)

head = reverseList(head)

print("\nLista encadeada invertida:")
printLinkedList(head)

... Lista encadeada original:
1 -> 2 -> 3 -> 4 -> 5 -> None

Lista encadeada invertida:
1 -> None
```

Código Inicial com Erro

Uso de IA (Prompt 1):

- **Pergunta:**

Eu tentei implementar a inversão de uma lista encadeada em Python, mas meu código só está retornando o primeiro elemento seguido de None. O que está errado na minha lógica de repetição? Aqui está o código:

- **Motivação:** Identificar o erro na lógica da função.
- **Resposta da IA:** Identificou que o return estava dentro do loop while, fazendo com que a função encerrasse na primeira iteração.



O erro no seu código é um detalhe sutil, mas muito comum em Python: a **indentação**.

O Diagnóstico

O seu comando `return anterior` está "dentro" do bloco do `while`.

Isso significa que, assim que o código entra no loop pela primeira vez (com o nó de valor 1), ele executa a inversão do primeiro ponteiro e encontra o `return`. Na mesma hora, a função para de funcionar e entrega o que tem em mãos. Por isso você só vê o `1 -> None`.

A Solução

Você precisa "tirar" o `return` de dentro do loop, para que ele só seja executado **depois** que o `while` terminar de percorrer toda a lista.

Veja como o bloco da função deve ficar:

Python



```
def reverseList(head):
    anterior = None
    atual = head

    while atual is not None:
        proximo = atual.next
        atual.next = anterior # Inverte o ponteiro
        anterior = atual      # Move o 'anterior' um passo à frente
        atual = proximo       # Move o 'atual' um passo à frente

    # O return deve estar alinhado com o 'while', e não dentro dele!
    return anterior
```

- **Avaliação:** A resposta foi correta e permitiu corrigir o problema movendo o `return` para fora do loop.

```

# Dado o 'head' de uma lista encadeada, inverter a lista e retornar *a lista invertida*.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

# criando a lista encadeada
head = ListNode(1)
head.next = ListNode(2)
head.next.next = ListNode(3)
head.next.next.next = ListNode(4)
head.next.next.next.next = ListNode(5)

# função para inverter
def reverseList(head):
    anterior = None
    atual = head

    while atual is not None:
        proximo = atual.next
        atual.next = anterior
        anterior = atual
        atual = proximo

    return anterior

def printLinkedList(head):
    while head is not None:
        print(head.val, end=" -> ")
        head = head.next
    print("None")

print("Lista encadeada original:")
printLinkedList(head)

head = reverseList(head)

print("\nLista encadeada invertida:")
printLinkedList(head)

*** Lista encadeada original:
1 -> 2 -> 3 -> 4 -> 5 -> None

Lista encadeada invertida:
5 -> 4 -> 3 -> 2 -> 1 -> None

```

Código Corrigido

Funcionamento da Solução:

A lógica da inversão ocorre da seguinte forma:

1. Guarda o próximo nó (proximo).
2. Inverte o ponteiro do nó atual.
3. Avança os ponteiros (anterior e atual).

Ao final do loop, anterior passa a ser o novo início da lista.

Identificando a Complexidade do Código:

Complexidade de Tempo

Como para inverter a lista inteira precisamos passar por cada índice apenas uma vez, caso a lista tenha cinco elementos, acontecerão cinco trocas, e assim por diante.

Assim, o tempo cresce na mesma proporção que o tamanho da lista, portanto, esta é a **Complexidade Linear ($O(n)$)**.

Complexidade de Memória

Como não foi criada uma lista nova, e sim apenas uma reorganização da lista inicial, foram necessárias apenas as três referências utilizadas: anterior, atual e próximo. Como o tamanho da lista não faria nenhuma diferença, o gasto de memória é fixo, portanto, esta é a **Complexidade Constante ($O(1)$)**.

Justificativa da Estrutura de Dados:

A lista encadeada é adequada para este problema, pois permite a manipulação direta dos ponteiros entre os nós.

Diferentemente de arrays, não é necessário deslocar elementos, tornando a operação de inversão mais eficiente em termos de manipulação estrutural.

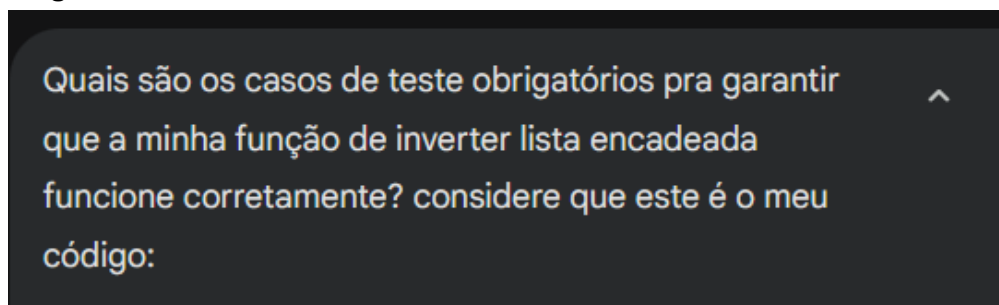
Testes:

Casos Especiais Considerados:

- Lista com múltiplos elementos.
- Lista com apenas um elemento.
- Lista vazia.

Uso de IA (Prompt 2):

- **Pergunta:**



Quais são os casos de teste obrigatórios pra garantir que a minha função de inverter lista encadeada funcione corretamente? considere que este é o meu código:

- **Motivação:** Garantir que a função funcione corretamente em diferentes cenários.
- **Resposta da IA:** Sugeriu três casos principais: lista normal, lista com um elemento e lista vazia.



Para garantir que sua função de inversão de lista encadeada seja robusta (o que os professores chamam de "código à prova de balas"), você precisa testar além do caso feliz de uma lista comum.

Aqui estão os 3 casos de teste obrigatórios para o seu relatório e para o seu código:

1. Lista com Múltiplos Elementos (O "Caso Feliz")

É o teste que você já fez (1 -> 2 -> 3 -> 4 -> 5). Ele garante que a lógica principal de percorrer a lista e inverter os ponteiros está funcionando para o fluxo normal.

- **Entrada:** 1 -> 2 -> 3 -> None
- **Saída Esperada:** 3 -> 2 -> 1 -> None

2. Lista com Apenas Um Elemento (Single Node)

Muitos algoritmos quebram aqui porque tentam acessar o `next` de algo que não existe. No seu código, o `while` deve rodar uma vez e retornar o próprio nó.

- **Entrada:** 1 -> None
- **Saída Esperada:** 1 -> None

3. Lista Vazia (Empty List/Edge Case)

Este é o teste mais importante para evitar o famoso erro `AttributeError: 'NoneType' object has no attribute 'next'`. Se a `head` for `None`, sua função deve retornar `None` graciosamente.

- **Entrada:** None
- **Saída Esperada:** None

✂ Como implementar esses testes no Colab?

Você pode adicionar uma célula com esta estrutura para validar tudo de uma vez:

Python

```
def test_reverse_list():
    # Caso 1: Lista Vazia
    assert reverseList(None) is None
    print("Teste Lista Vazia: OK")

    # Caso 2: Um único elemento
    single_node = ListNode(10)
    res_single = reverseList(single_node)
    assert res_single.val == 10 and res_single.next is None
    print("Teste Um Elemento: OK")

    # Caso 3: Lista Normal (reutilizando a lógica de inverter duas vezes)
    head_test = ListNode(1, ListNode(2))
    # Inverte e depois inverte de novo para voltar ao original
    inv = reverseList(head_test)
    assert inv.val == 2 and inv.next.val == 1
    print("Teste Lista Normal: OK")

test_reverse_list()
```

- **Avaliação:** As sugestões foram adequadas e utilizadas na criação dos testes.

```
def test_reverse_list():  
    # Caso 1: Lista Vazia  
    assert reverselist(None) is None  
    print("Teste Lista Vazia: OK")  
  
    # Caso 2: Um único elemento  
    single_node = ListNode(10)  
    res_single = reverselist(single_node)  
    assert res_single.val == 10 and res_single.next is None  
    print("Teste Um Elemento: OK")  
  
    # Caso 3: Lista Normal (reutilizando a lógica de inverter duas vezes)  
    head_test = ListNode(1, ListNode(2))  
    # Inverte e depois inverte de novo para voltar ao original  
    inv = reverselist(head_test)  
    assert inv.val == 2 and inv.next.val == 1  
    print("Teste Lista Normal: OK")  
  
test_reverse_list()  
  
... Teste Lista Vazia: OK  
    Teste Um Elemento: OK  
    Teste Lista Normal: OK
```

Código de Testes

Remove Nth Node From End of List

- Plataforma: LeetCode
- Link: [Remove Nth Node From End of List - LeetCode](#)
- Problema de Lista Encadeada

Descrição do Problema:

Dado o nó inicial (head) de uma lista encadeada, o objetivo é remover o n-ésimo nó a partir do final da lista e retornar o novo início da lista.

Estratégia da Solução:

A solução foi implementada utilizando dois ponteiros (fast e slow) e um nó auxiliar (dummy).

O ponteiro fast é avançado n+1 posições à frente do slow criando uma distância fixa entre eles. Em seguida, ambos avançam simultaneamente até que o fast atinja o final da lista.

Desta forma, o ponteiro slow estará posicionado imediatamente antes do nó que deve ser removido.

```
# Dado o `head` de uma lista encadeada, remover o `n`-ésimo nó do final da lista e retornar o `head`
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def removeNthFromEnd(head, n):
    dummy = ListNode(0)
    dummy.next = head

    slow = dummy
    fast = dummy

    for _ in range(n+1):
        fast = fast.next

    while fast:
        slow = slow.next
        fast = fast.next

    slow.next = slow.next.next

    return dummy.next
```

```

▶ # Testando
head = ListNode(1, ListNode(2, ListNode(3, ListNode(4, ListNode(5)))))
n = 2

def printLinkedList(head):
    current = head
    while current is not None:
        print(current.val, end=" -> ")
        current = current.next
    print("None")

print("Lista Original:")
printLinkedList(head)

head = removeNthFromEnd(head, n)

print("\nLista Modificada:")
printLinkedList(head)

... Lista Original:
1 -> 2 -> 3 -> 4 -> 5 -> None

Lista Modificada:
1 -> 2 -> 3 -> 5 -> None

```

Funcionamento da Solução:

A solução utiliza dois ponteiros para percorrer a lista:

- Um nó auxiliar (dummy) é criado para facilitar a remoção, especialmente quando o primeiro elemento precisa ser removido.
- O ponteiro fast avança $n+1$ posições à frente.
- Os ponteiros fast e slow avançam juntos até fast chegar ao final.
- O ponteiro slow aponta para o nó anterior ao que será removido.
- A remoção é feita ajustando o ponteiro: `slow.next = slow.next.next`

Justificativa da Estrutura de Dados:

A lista encadeada é adequada para este problema pois ela permite a remoção de elementos apenas ajustando ponteiros, sem a necessidade do deslocamento de dados.

O uso de dois ponteiros permite resolver o problema em apenas uma passagem pela lista, o que torna a solução mais eficiente.

Identificando a Complexidade do Código:

Complexidade de Tempo

Como a lista é percorrida apenas uma vez, utilizando a técnica de dois ponteiros, a solução possui uma **Complexidade Linear ($O(n)$)**.

Complexidade de Memória

Apesar da criação de um nó extra (o dummy) e os ponteiros, a quantidade de memória adicional é fixa. Não importa a quantidade de elementos na lista, o código sempre usará apenas o mesmo conjunto de referências auxiliares para ajustar os ponteiros existentes. Como o gasto de memória não depende do tamanho da entrada, esta é a **Complexidade Constante ($O(1)$)**.

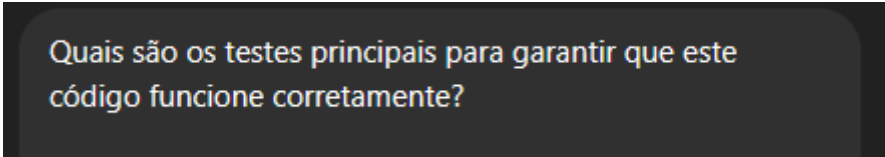
Testes:

Casos Especiais Considerados:

- Remoção de um elemento no meio da lista.
- Remoção do último elemento da lista.
- Remoção do primeiro elemento da lista.
- Lista com apenas um elemento.
- Lista com dois elementos.
- Caso onde o n é igual ao tamanho da lista.

Uso de IA (Prompt 1):

- **Pergunta:**



Quais são os testes principais para garantir que este código funcione corretamente?

- **Motivação:** Garantir que a função funcione corretamente em diferentes cenários, incluindo casos extremos.
- **Resposta da IA:**

Testes são basicamente garantir que seu código funciona em **casos normais** + **casos extremos (edge cases)**.

Vou te mostrar os principais testes que você **PRECISA** cobrir 📌

✅ 1. Caso normal (meio da lista)

👉 Já é o que você fez:

Python



▶ Executar

```
# 1 -> 2 -> 3 -> 4 -> 5, n = 2  
# remove 4
```

✔ Esperado:

1 -> 2 -> 3 -> 5



✅ 2. Remover o ÚLTIMO nó

Python



▶ Executar

```
# 1 -> 2 -> 3 -> 4 -> 5, n = 1
```

✔ Esperado:

1 -> 2 -> 3 -> 4



💡 Testa se o final da lista está sendo tratado corretamente.

✓ 3. Remover o PRIMEIRO nó (super importante!)

</> Python



▶ Executar

```
# 1 -> 2 -> 3 -> 4 -> 5, n = 5
```

✓ Esperado:

```
2 -> 3 -> 4 -> 5
```



✨ Esse é o caso que quebra soluções sem `dummy`.

✓ 4. Lista com UM único elemento

</> Python



▶ Executar

```
# 1, n = 1
```

✓ Esperado:

```
None
```



💡 Aqui a lista fica vazia.

✓ 5. Lista com DOIS elementos

Python



▶ Executar

```
# 1 -> 2, n = 1
```

✓ Esperado:

1



E também:

Python



▶ Executar

```
# 1 -> 2, n = 2
```

✓ Esperado:

2



✓ 6. n igual ao tamanho da lista

(é tipo o caso 3, mas importante pensar nisso explicitamente)

Python



▶ Executar

```
# 10 -> 20 -> 30, n = 3
```

✓ Esperado:

20 -> 30



7. (Opcional) n inválido

Depende do problema, mas bom pensar:

 Python



 Executar

```
# n maior que o tamanho
```

Seu código atual pode dar erro aqui.

COMO TESTAR DE FORMA ORGANIZADA

Você pode montar vários testes assim:

 Python



 Executar

```
def testar(head, n):  
    print("Original:")  
    printLinkedList(head)  
  
    head = removeNthFromEnd(head, n)  
  
    print("Resultado:")  
    printLinkedList(head)  
    print("_" * 30)
```

- **Avaliação:** A resposta foi adequada e contribuiu para a criação de uma série de testes mais completa, e que garante uma maior confiabilidade da solução.
- **Resultado dos Testes:** Todos os testes executados retornaram os resultados esperados, indicando que a função está funcionando corretamente para os diferentes cenários considerados.

```

▶ print("Teste de remover o ultimo nó (n=1)\n")
head2 = ListNode(1, ListNode(2, ListNode(3, ListNode(4, ListNode(5)))))
print("Lista Original: ")
printLinkedList(head2)

resultado2 = removeNthFromEnd(head2, 1)

print("Lista Modificada: ")
printLinkedList(resultado2)

print("\n")

print("Teste de remover o primeiro nó (n=5)\n")
head3 = ListNode(1, ListNode(2, ListNode(3, ListNode(4, ListNode(5)))))
print("Lista Original: ")
printLinkedList(head3)

resultado3 = removeNthFromEnd(head3, 5)

print("Lista Modificada:")
printLinkedList(resultado3)

print("\n")

print("Teste de lista com um único elemento (n=1)\n")
head4 = ListNode(1)
print("Lista Original: ")
printLinkedList(head4)

resultado4 = removeNthFromEnd(head4, 1)

print("Lista Modificada:")
printLinkedList(resultado4)

```

```

▶ print("\n")

print("Teste de n igual ao tamanho da lista (n=3)\n")
head5 = ListNode(10, ListNode(20, ListNode(30)))
print("Lista Original: ")
printLinkedList(head5)

resultado5 = removeNthFromEnd(head5, 3)

print("Lista Modificada:")
printLinkedList(resultado5)

```

```
Teste de remover o ultimo nó (n=1)
...
Lista Original:
1 -> 2 -> 3 -> 4 -> 5 -> None
Lista Modificada:
1 -> 2 -> 3 -> 4 -> None

Teste de remover o primeiro nó (n=5)

Lista Original:
1 -> 2 -> 3 -> 4 -> 5 -> None
Lista Modificada:
2 -> 3 -> 4 -> 5 -> None

Teste de lista com um único elemento (n=1)

Lista Original:
1 -> None
Lista Modificada:
None

Teste de n igual ao tamanho da lista (n=3)

Lista Original:
10 -> 20 -> 30 -> None
Lista Modificada:
20 -> 30 -> None
```

Linked List Cycle

- Plataforma: LeetCode
- Link: [Linked List Cycle - LeetCode](#)
- Problema de Lista Encadeada

Descrição do Problema:

Esse problema envolve verificar se uma lista encadeada possui um ciclo, ou seja, se algum nó aponta para um nó anterior na estrutura. A lista encadeada é essencial aqui, pois o desafio depende da navegação entre os nós por meio de ponteiros. A solução mais eficiente utiliza dois ponteiros (lento e rápido), explorando a estrutura dinâmica da lista sem necessidade de memória adicional, com complexidade $O(n)$.

Estratégia da Solução:

A estratégia utilizada no seu código é baseada na **detecção de repetição de nós já visitados**. Ao percorrer uma lista encadeada **sem ciclo**, cada nó é visitado apenas uma vez, mas se existir um ciclo, em algum momento você vai voltar para um nó que já foi visitado antes.

```
# Sem IA
from typing import Optional

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        visitados = []

        atual = head
        while atual is not None:
            if atual in visitados:
                return True

            visitados.append(atual)
            atual = atual.next

        return False
```

Código inicial com erro

Uso de IA:

- **Pergunta:** Dado o problema [...] e exemplos descritos, o código escrito por mim segue os parâmetros necessários?
- **Motivação:** Identificar o erro na lógica da função.
- **Resposta da IA:**

⚠ Problema principal: eficiência

1. Complexidade de tempo

Python

```
if atual in visitados:
```



▶ Executar

Isso é $O(n)$, porque a lista precisa ser percorrida.

Como isso acontece dentro de um loop, o total vira:

👉 $O(n^2)$ no pior caso

2. Uso de memória

Você armazena todos os nós:

👉 $O(n)$ de espaço

- **Avaliação:** Junto com o problema apontado a IA também trouxe uma solução para resolução, o que logo foi comprovada ser a mais eficaz.
 - A solução detecta ciclos armazenando nós visitados em um **set**.
 - Se um nó for encontrado novamente, há ciclo.
 - Complexidade de tempo: **$O(n)$** .
 - Complexidade de memória: **$O(n)$** .
 - O uso de **set** garante eficiência nas operações de busca e inserção.

💡 Como melhorar (sem mudar muito sua ideia)

Trocar a lista por um **set** :

Python

```
visitados = set()
```



▶ Executar

Porque:

- **in** em set é **$O(1)$**
- Mantém a mesma lógica

```

from typing import Optional

class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        visitados = set()

        atual = head
        while atual is not None:
            if atual in visitados:
                return True

            visitados.add(atual)
            atual = atual.next

        return False

```

Código corrigido

Funcionamento da Solução:

A solução percorre a lista encadeada a partir do nó inicial (head), utilizando um conjunto (set) para armazenar todos os nós já visitados.

Durante a travessia:

- Para cada nó atual, é verificado se ele já está no conjunto visitados.
- Se estiver, significa que o algoritmo voltou a um nó previamente visitado, caracterizando um ciclo na lista → retorna True.
- Caso contrário, o nó é adicionado ao conjunto e o algoritmo segue para o próximo nó (atual.next).

Se o percurso chegar ao final da lista (None), significa que não há ciclo → retorna False.

Identificando a Complexidade do Código:

Complexidade de Tempo: $O(n)$

Cada nó da lista é visitado no máximo uma vez.

As operações de verificação (in) e inserção (add) em um set possuem custo médio de $O(1)$. Portanto, o tempo total cresce linearmente com o número de nós.

Complexidade de Memória $O(n)$

O conjunto visitados armazena todos os nós percorridos. No pior caso (sem ciclo), todos os n nós são armazenados.

Justificativa da Estrutura de Dados:

A estrutura de dados escolhida foi um **conjunto (set)**, pois oferece:

- **Busca eficiente ($O(1)$):** permite verificar rapidamente se um nó já foi visitado.
- **Inserção eficiente ($O(1)$):** adiciona novos nós sem custo significativo.
- **Garantia de unicidade:** evita duplicatas automaticamente, facilitando a detecção de repetição.

Essa escolha torna o algoritmo mais eficiente do que usar uma lista, que teria custo $O(n)$ para busca, resultando em uma complexidade total pior ($O(n^2)$).

Testes:

Casos Especiais Considerados:

- Lista com ciclo no meio.
- Lista com ciclo no início.
- Lista sem ciclo.
- Lista maior sem ciclo.
- Lista vazia (caso de borda).
- Ciclo no próprio nó.

PROBLEMS 2

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

SPELL CHECKER

133

```
C:\Users\silva\Desktop\code-review\Code-Interview>C:\Users\silva\AppData\Local\Python\bin\python.exe c:/Users/silva/Desktop/code-review/Code-Interview/mariele/teste-lista-encadeada.py
Ciclo no meio: OK
Ciclo no início: OK
Sem ciclo: OK
Lista maior sem ciclo: OK
Lista vazia: OK
Ciclo no próprio nó: OK
```

Daily Temperatures

- Plataforma: LeetCode
- Link: [Daily Temperatures - LeetCode](#)
- Problema de Pilha

Descrição do Problema:

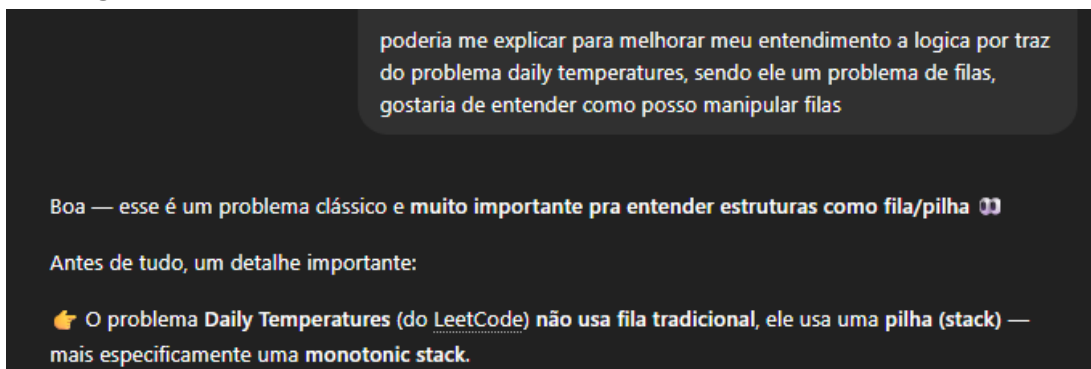
Dado uma lista de números inteiros representando a temperatura máxima alcançada em determinado dia, deve-se entregar outra lista contendo o número de dias que se deve esperar, a partir do dia atual, para chegar em um dia mais quente.

Estratégia da Solução:

Fazer uso de uma fila auxiliar para armazenar as operações realizadas subtraindo seu número de comparações, assim para cada dia que for retirado da fila será atribuído +1 ponto até que se encontre um dia com a temperatura maior, e caso não seja encontrado, essa pontuação será zerada enfatizando que não há mais dias mais quentes que o atual.

Uso de IA (Prompt 1):

- Pergunta:



- **Motivação:** Não entendi corretamente a proposta do desafio, sei que se trata de manipulação de pilhas (no caso eu errei no prompt e me referi a filas), mas eu tinha dúvidas de como eu poderia manipulá-las.
- **Resposta da IA:** Ela me explicou melhor a lógica por trás do desafio e me deu exemplos de como realizar a manipulação de pilhas em Python.

  **O que o problema pede?**

Dado um array de temperaturas:

```
Python
[73, 74, 75, 71, 69, 72, 76, 73]
```

Você precisa retornar:

```
Python
[1, 1, 4, 2, 1, 1, 0, 0]
```

👉 Cada posição diz:
quantos dias você precisa esperar até encontrar uma temperatura maior

  **Intuição (a chave do problema)**

Para cada dia, você quer saber:

"Quando vai aparecer uma temperatura maior que a atual?"

 **Forma ingênua ($O(n^2)$)**

```
Python
para cada dia:
    olhar todos os próximos dias
```

Muito lento.

  **Ideia eficiente: usar uma pilha**

A pilha vai guardar índices dos dias que ainda não encontraram uma temperatura maior.

 **Como a pilha funciona aqui** 

Você percorre o array uma vez:

- **Avaliando Resposta:** Excelente novamente, graças ao chat tive um entendimento da proposta do desafio e como realizar a manipulação de pilhas.

```

class Solution(object):
    def dailyTemperatures(self, temperatures):
        """
        :type temperatures: List[int]
        :rtype: List[int]
        """
        n = len(temperatures)
        answer = [0] * n
        stack = []

        for i in range(n):
            while stack and temperatures[i] > temperatures[stack[-1]]:
                prev = stack.pop()
                answer[prev] = i - prev

            stack.append(i)

        return answer

```

Primeira Versão Funcional do Código

Justificativa da Estrutura de Dados:

- stack é a pilha que armazenará os dias faltantes para o próximo mais quente.
- while é o loop que repetirá enquanto o stack e a temperatura do dia atual for menor que a temperatura na última posição da fila de dia que ainda não foram comparados.
- prev é a variável que armazenará o último valor adicionado em stack ao mesmo tempo em que o remove e substitui pela equação (maior temperatura atual - maior temperatura anterior) esse processo se repete dentro do while até que seja falso, e a posição da nova maior temperatura seja guardada dentro da pilha.

Identificando a Complexidade do Código:

Complexidade de Tempo:

Mesmo contendo um for e um while, cada elemento entra e sai da lista apenas uma vez, evitando retrabalho, fazendo com que a complexidade do código seja linear (**O(n)**).

Complexidade de Memória:

Mesmo que a operação nunca entre no while, o uso de memória nunca superará a quantidade já utilizada, e como é desconsiderado constantes a complexidade se mantém linear (**O(n)**).

Testes:

```

Teste com temperaturas negativas:
temperaturas: [-1, -2, -3, -1, -5]
output obtido: [0, 2, 1, 0, 0]

```

Teste com string:

temperaturas: ['30', '40', '50', '20', '10', '25', '60', '35']

output obtido: [1, 1, 4, 2, 1, 1, 0, 0]

Teste com temperatura nunca sendo maior:

temperaturas: [100, 90, 80, 70, 60, 50, 40, 30]

output obtido: [0, 0, 0, 0, 0, 0, 0, 0]

- **Conclusão:** Todos os teste foram um sucesso, mesmo com sting o código ainda identifica os valores, mas caso fossem usados letras ele os compararia por ordem alfabética, também é mostrado um dos piores caso para uso de memória, onde o stack nunca é removido uma unidade da pilha.

Simplify Path

- Plataforma: LeetCode
- Link: [Simplify Path - LeetCode](#)
- Problema de Pilha

Descrição do Problema:

Dado um caminho absoluto de um sistema de arquivos no estilo Unix, o objetivo é transformar este caminho absoluto em seu caminho canônico simplificado.

O caminho poderá conter:

- Um único ponto . , que representa o diretório atual.
- Ponto duplo . . , que representa o diretório pai.
- Múltiplas barras consecutivas, como // e /// , que serão tratadas como uma única barra / .
- Sequências de pontos que não correspondem às outras regras, por exemplo, ... e devem ser tratadas como um nome válido de diretório ou arquivo.

Já o caminho canônico simplificado deve seguir estas regras:

- O caminho deve começar com uma única barra / .
- Os diretórios dentro do caminho devem ser separados por uma única barra / .
- O caminho não deve terminar com uma barra / a menos que seja o diretório raiz.
- O caminho não deve conter pontos simples ou duplos (. e . .) usados para denotar diretórios atuais ou diretórios pais.

Estratégia da Solução:

A solução foi implementada utilizando uma pilha (stack) para simular a navegação entre diretórios em um sistema de arquivos Unix.

O caminho é inicialmente dividido em partes usando / como separador e, em seguida, cada elemento é processado de acordo com as regras do problema:

- " " ou . são ignorados.
- . . remove o último diretório da pilha (se este existir).
- Outros valores são adicionados à pilha.

No final, os diretórios válidos armazenados na pilha são utilizados para reconstruir o caminho simplificado.

```
# Transformar o caminho de arquivos no sistema Unix para o simplificado
def simplificarCaminho(caminho):
    stack = []
    partes = caminho.split('/')

    for parte in partes:
        if parte == '' or parte == '.':
            continue
        if parte == '..':
            if stack:
                stack.pop()
            else:
                stack.append(parte)
        else:
            stack.append(parte)

    return "/" + "/".join(stack)

print(simplificarCaminho("/home/"))
print(simplificarCaminho("/home//foo/"))
print(simplificarCaminho("/home/user/Documents/../Pictures"))
print(simplificarCaminho("/../"))
print(simplificarCaminho("/.../a/..b/c/..d/.."))

... /home
    /home/foo
    /home/user/Pictures
    /
    /.../b/d
```

Funcionamento da Solução:

A solução utiliza uma pilha para controlar os diretórios do caminho:

- Cada parte do caminho é analisada separadamente após o uso do `split('/')`.
- Quando um diretório válido é encontrado, ele é adicionado à pilha.
- Quando o símbolo `..` é encontrado, o último diretório inserido é removido da pilha, caso ele exista.
- Os elementos `.` e partes vazias são ignorados.

No final, os elementos restantes na pilha representam o caminho correto sendo então unidos para formar o caminho simplificado.

Justificativa da Estrutura de Dados:

A pilha é adequada para este problema pois permite desfazer operações recentes de forma eficiente.

Como o comando `..` indica retorno ao diretório anterior, a remoção do último elemento inserido na pilha representa exatamente esse comportamento.

Identificando a Complexidade do Código:

Complexidade de Tempo

Para simplificar o caminho, o código precisa executar três passos principais: separar a string original em partes (utilizando a barra / como guia), e então percorrer cada uma

dessas partes uma única vez para decidir se deve empilhar ou desempilhar um nome, e, então, juntar tudo novamente.

Como cada caractere e cada diretório são processados apenas uma vez, o tempo de execução cresce na mesma proporção que o tamanho do caminho fornecido. Portanto, esta é a **Complexidade Linear ($O(n)$)**.

Complexidade de Tempo

Ao contrário de problemas em que o gasto de memória é fixo, neste problema, é necessário uma estrutura auxiliar, a pilha, para armazenar os nomes dos diretórios.

Caso o caminho for muito longo e cheio de pastas novas (/a/b/c/d..., por exemplo), a pilha precisará guardar todos os nomes simultaneamente. Além disso, a lista gerada ao separar a string também ocupa espaço, proporcional à entrada.

Como o gasto de memória aumenta conforme o tamanho do caminho cresce, esta é a **Complexidade Linear ($O(n)$)**.

Assim, é possível dizer que o código é equilibrado, pois mantém uma execução rápida (Tempo), porém exige um pouco mais de espaço para organizar os dados (Memória).

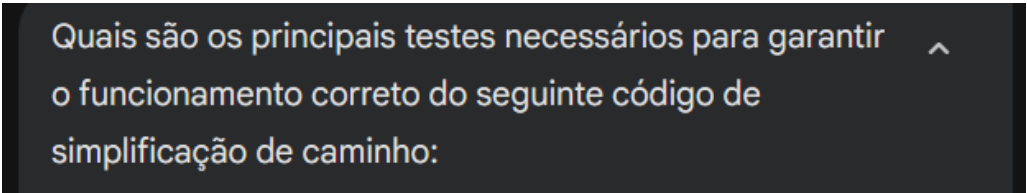
Testes:

Casos Especiais Considerados:

- Caminho raiz (/).
- Tentativa de subir além da raiz (/ . . /).
- Múltiplas barras (/ /)..
- Uso de . (diretorio atual).
- Uso de . . (diretorio pai).
- Nomes com múltiplos pontos (. . .).
- Caminhos complexos combinando diferentes regras.

Uso de IA (Prompt 2):

- **Pergunta:**



Quais são os principais testes necessários para garantir o funcionamento correto do seguinte código de simplificação de caminho: ^

- **Motivação:** Garantir que a função seja validada em diferentes cenários.
- **Resposta da IA:** A IA sugeriu a criação de testes envolvendo:
 - Caminho raiz.
 - Subida além da raiz.
 - Múltiplas barras consecutivas.
 - Uso de "." e ". .".
 - Nomes com múltiplos pontos.
 - Caminhos mais complexos combinando diferentes regras.



Para garantir que o seu problema de nível **Médio** esteja realmente correto e robusto para o relatório, você precisa testar o que chamamos de "casos de borda" (edge cases). Diferente dos arrays, caminhos de sistema de arquivos têm várias pegadinhas de símbolos.

Aqui estão os 5 principais testes que não podem faltar:

1. O "Salto no Vazio" (Subir além da Raiz)

Cenário: O que acontece se o usuário tentar voltar um nível (`..`) estando na pasta raiz `/` ?

- **Input:** `"/../../"`
- **Expectativa:** O código não deve dar erro de "pilha vazia" (Empty Stack) e deve retornar apenas `/`. Seu código já trata isso com o `if stack: stack.pop()`.

2. Barras Consecutivas (Múltiplas Barras)

Cenário: Caminhos que vêm com erros de digitação ou protocolos diferentes, como `//` ou `///`.

- **Input:** `"/home//foo/"`
- **Expectativa:** O `split('/')` vai gerar strings vazias (`''`). O seu código deve ignorar essas partes e retornar `/home/foo`.

3. O Ponto Solitário (Diretório Atual)

Cenário: O uso do `.` que, no Unix, significa "fique onde você está".

- **Input:** `"/a/. /b/. /c/. /"`
- **Expectativa:** Os pontos devem ser completamente descartados. O resultado deve ser `/a/b/c`.

4. Nomes com Múltiplos Pontos (Arquivos vs. Comandos)

Cenário: Como o código diferencia o comando `..` de um nome de pasta chamado `...` ou `..hidden`.

- **Input:** `"/../"` ou `"/..hidden/"`
- **Expectativa:** Como apenas `..` é o comando de subir, qualquer outra combinação de pontos deve ser tratada como um nome de pasta comum. O resultado deve ser `/...` ou `/..hidden`.

5. Caminho Vazio ou Apenas Raiz

Cenário: Entradas mínimas.

- **Input:** `"/"`
- **Expectativa:** Deve retornar sempre pelo menos a barra inicial `/`.

🔧 Implementação dos Testes no Colab

Você pode usar este bloco para deixar seus testes automatizados e "limpos" para o relatório:

Python

```
def test_simplificar_caminho():
    casos = [
        ("/", "/"),
        ("/../", "/"),
        ("/home//foo/", "/home/foo"),
        ("/a/./b/../../c/", "/c"),
        ("/.../a/./b/c/./d/./", "/.../b/d")
    ]

    for entrada, esperado in casos:
        resultado = simplificarCaminho(entrada)
        status = "✅ PASSOU" if resultado == esperado else "❌ FALHO"
        print(f"Entrada: {entrada:25} | Esperado: {esperado:10} | Resultado: {resultado:10} | Status: {status}")

test_simplificar_caminho()
```

- **Avaliação:** A resposta foi adequada e contribuiu para a criação de uma bateria de testes mais completa, garantindo maior confiabilidade da solução.
- **Resultado dos Testes:** Todos os testes executados retornaram os resultados esperados, indicando que a função está correta para os diferentes cenários analisados.

```
def test_simplificar_caminho():
    casos = [
        ("/", "/"),
        ("/../", "/"),
        ("/home//foo/", "/home/foo"),
        ("/a/./b/../../c/", "/c"),
        ("/.../a/./b/c/./d/./", "/.../b/d")
    ]

    for entrada, esperado in casos:
        resultado = simplificarCaminho(entrada)
        status = "PASSOU" if resultado == esperado else "FALHO"
        print(f"Entrada: {entrada:25} | Esperado: {esperado:10} | Resultado: {resultado:10} | Status: {status}")

test_simplificar_caminho()
```

Entrada: /	Esperado: /	Resultado: /	Status: PASSOU
Entrada: /../	Esperado: /	Resultado: /	Status: PASSOU
Entrada: /home//foo/	Esperado: /home/foo	Resultado: /home/foo	Status: PASSOU
Entrada: /a/./b/../../c/	Esperado: /c	Resultado: /c	Status: PASSOU
Entrada: /.../a/./b/c/./d/./	Esperado: /.../b/d	Resultado: /.../b/d	Status: PASSOU

Valid Parentheses

- Plataforma: LeetCode
- Link: [Valid Parentheses - LeetCode](#)
- Problema de Pilha

Descrição do Problema:

Dada uma string *s* contendo apenas os caracteres '(', ')', '{', '}', '[' e ']', determine se a string de entrada é válida.

Estratégia da Solução:

A pilha é ideal para esse problema porque segue o princípio LIFO (Last In, First Out). Ao percorrer a string, cada caractere de abertura é empilhado. Quando aparece um caractere de fechamento, verifica-se se ele corresponde ao último elemento inserido na pilha. Isso garante a validação correta da ordem e do balanceamento dos parênteses.

```
# Solução sem IA
class Solution:
    def isValid(self, s: str) -> bool:
        mudou = True

        while mudou:
            mudou = False

            if "()" in s:
                s = s.replace("()", "")
                mudou = True

            if "[]" in s:
                s = s.replace("[]", "")
                mudou = True

            if "{}" in s:
                s = s.replace("{} ", "")
                mudou = True

        return s == ""
```

Uso de IA:

- **Pergunta:** “Dado esse problema [...], cujas restrições são [...]. O código abaixo escrito por mim resolve corretamente o problema?”.

✓ Por que funciona?

A ideia que você usou é bem inteligente:

- Você remove pares válidos `()`, `[]`, `{}` repetidamente
- Continua fazendo isso enquanto ainda houver mudanças na string
- No final:
 - Se a string ficou vazia → **válida**
 - Se sobrou algo → **inválida**

🧠 Intuição por trás

Isso simula indiretamente uma **pilha (stack)**:

- Pares internos são removidos primeiro
- Depois os externos vão sendo resolvidos

- **Resposta da IA:** Apesar do meu código estar correto, a IA me informou que existe uma solução bem mais utilizada e que a minha não era a mais eficiente.

💡 Forma mais comum (com pilha - $O(n)$)

Só pra comparar, a solução mais usada é com stack:

```
Python
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []
        pares = {'(': ')', '[': ']', '{': '}'

        for c in s:
            if c in pares:
                if not stack or stack[-1] != pares[c]:
                    return False
                stack.pop()
            else:
                stack.append(c)

        return len(stack) == 0
```

- **Avaliando Resposta:** As respostas da IA são bem úteis para comparação de código, com ela é possível ter a dimensão do nosso nível de programação e raciocínio lógico, além de termos acesso a técnicas e soluções mais eficientes e profissionais.

Justificativa da Estrutura de Dados:

A estrutura de dados utilizada na solução otimizada é a **pilha (stack)**, implementada com uma lista em Python.

A escolha da pilha é adequada porque o problema exige validar **ordem e correspondência** dos parênteses. A pilha segue o princípio **LIFO (Last In, First Out)**, ou seja, o último

elemento inserido é o primeiro a sair, o que combina perfeitamente com o comportamento dos parênteses:

- Ao encontrar um símbolo de abertura ((, [, {), ele é **empilhado**.
- Ao encontrar um símbolo de fechamento (),], }), verificamos se ele corresponde ao **último símbolo aberto**.
- Se corresponder, removemos da pilha.
- Caso contrário, a sequência é inválida.

Essa abordagem garante que:

- A ordem dos fechamentos seja respeitada.
- Cada abertura tenha um fechamento correspondente correto.

Identificando a Complexidade do Código:

Complexidade de Tempo: $O(n)$

- O algoritmo percorre a string **uma única vez**.
- Cada caractere é processado apenas uma vez.
- Operações de pilha (append e pop) são $O(1)$.

Portanto, o tempo cresce linearmente com o tamanho da entrada.

Complexidade de Memória: $O(n)$

No pior caso (ex: " ((((["), todos os caracteres são armazenados na pilha. Logo, o uso de memória cresce proporcionalmente ao tamanho da string.

Testes:

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 182 +
C:\Users\silva\Desktop\code-review\Code-Interview>C:\Users\silva\AppData\Local\Python\bin\python.exe c:/Users/silva/Desktop/code-review/Code-Interview/mariele/test_valid_parentheses.py
=== TESTANDO SOLUÇÃO REPLACE ===
Entrada: () | Esperado: True | Resultado: True
Entrada: ()[]{} | Esperado: True | Resultado: True
Entrada: [] | Esperado: False | Resultado: False
Entrada: [[]] | Esperado: True | Resultado: True
Entrada: [[]] | Esperado: False | Resultado: False
Entrada: | Esperado: True | Resultado: True
Entrada: ((((( | Esperado: False | Resultado: False
Entrada: ))))) | Esperado: False | Resultado: False
Entrada: {[()()]} | Esperado: True | Resultado: True
Entrada: {[()]} | Esperado: False | Resultado: False

=== TESTANDO SOLUÇÃO STACK ===
Entrada: () | Esperado: True | Resultado: True
Entrada: ()[]{} | Esperado: True | Resultado: True
Entrada: [] | Esperado: False | Resultado: False
Entrada: [[]] | Esperado: True | Resultado: True
Entrada: [[]] | Esperado: False | Resultado: False
Entrada: | Esperado: True | Resultado: True
Entrada: ((((( | Esperado: False | Resultado: False
Entrada: ))))) | Esperado: False | Resultado: False
Entrada: {[()()]} | Esperado: True | Resultado: True
Entrada: {[()]} | Esperado: False | Resultado: False

C:\Users\silva\Desktop\code-review\Code-Interview>
```

- **Conclusão:** Todos os testes foram feitos e executados sem erro.